

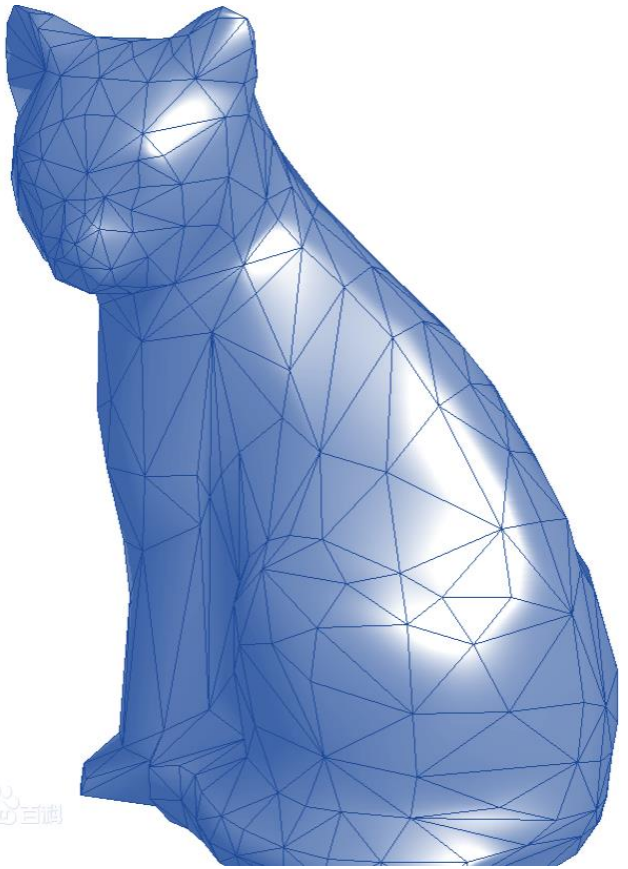
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

第4章 STL模型切片轮廓计算基础

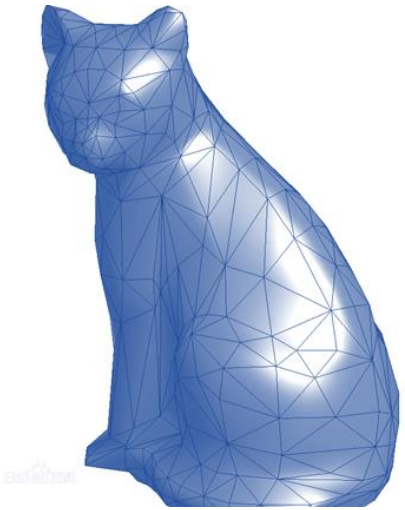
- STL模型加载
- 空间三角形和平面求交算法
- STL模型截交计算
- 截交线段拼接计算
- 轮廓方向识别与调整
- 切片数据存取
- SliceModel模块



1. STL模型加载

• STL模型文件解析

- STL（Stereo Lithography）被认为是三维打印模型的“**事实标准**”，是由3D systems公司于1988年制定的一个接口协议。
- STL模型由一系列三角面片组成，相邻三角面片之间遵守“**点对点、边对边**”的规则。如果一个三角形的顶点落在另一个三角形的边上，那么这个STL模型就可被认为是非法的。
- STL文件有两种保存形式，一种以**文本（ASCII）**形式保存坐标，另一种以**二进制（Binary）**形式保存坐标。文本形式的STL适合阅读，但从文件尺寸上来说，对同一模型，用二进制存储显然要比用文本存储更节省空间。



```
facet normal 0.000000e+000 -9.807748e-001 -1.951428e-001   → 法向
  outer loop                                                          第一个面片
    vertex 6.000000e+001 -5.323471e+001 -1.364642e+002      → 顶点1
    vertex 0.000000e+000 -5.323471e+001 -1.364642e+002      → 顶点2
    vertex 0.000000e+000 -5.306457e+001 -1.373193e+002      → 顶点3
  endloop
endfacet
facet normal 0.000000e+000 4.805336e-001 8.769763e-001              第二个面片
  outer loop
    vertex 6.000000e+001 -5.196816e+000 -1.107972e+002
    vertex 0.000000e+000 -5.946526e+000 -1.103820e+002
    vertex 6.000000e+001 -5.946526e+000 -1.103820e+002
  endloop
endfacet
facet normal 0.000000e+000 4.638007e-001 8.859395e-001              第三个面片
  outer loop
    vertex 6.000000e+001 -5.946526e+000 -1.103820e+002
    vertex 0.000000e+000 -5.946526e+000 -1.103820e+002
    vertex 0.000000e+000 -6.858338e+000 -1.099109e+002
  endloop
endfacet
⋮
```

- 每个面片的第1行由“facet normal”关键词引导，后面存储面片法向量坐标（数据用科学计数法表示，e表示10的指数）；
- 第3、4、5行由“vertex”关键词引导，后面分别存储三角面片3个顶点坐标。
- 在STL中，各个面片是无序保存的，面片与面片之间不存在任何逻辑关系。

文本类型的STL文件数据存储形式

1. STL模型加载

- **STL模型文件读取**
- 要在计算机内存上保存一个STL文件，首先需要建立三角面片数据结构（**Triangle**类）。

Triangle.py	三角形类，封装了3个顶点坐标和1个法向量坐标	4-1

```
1   from GeomBase import *
2   class Triangle:
3       def __init__(self, A, B, C, N = Vector3D(0,0,0)):
4           self.A, self.B, self.C, self.N = A.clone(), B.clone(), C.clone(), N.clone()    # 顶点和法向
5           self.zs = []                                                                    # 存储面片包含的层高
6       def __str__(self):
7           pass
8       def zMinPnt(self):                                                                  # 3个顶点中的Z方向最低点
9           pass
10      def zMaxPnt(self):                                                                  # 3个顶点中的Z方向最高点
11          pass
```

- **__init__**函数参数为3个顶点和1个法向量，法向量默认为零向量。
- 属性**A、B、C、N**分别为三角形三个顶点和法向量。
- 属性**zs**保存面片所包含的层高数值，这个属性我们后面会用到，这里暂不做讨论。
- **zMinPnt**函数用于获取三角形3个顶点中在Z方向上的最低点，与之对应的**zMaxPnt**函数用于获取Z方向上的最高点。

1. STL模型加载

- **STL模型文件读取**
- 为读取STL模型，专门建立StlModel类，并在其中封装相关函数。

StlModel.py	STL模型类，主要封装了STL文件读取功能	4-2

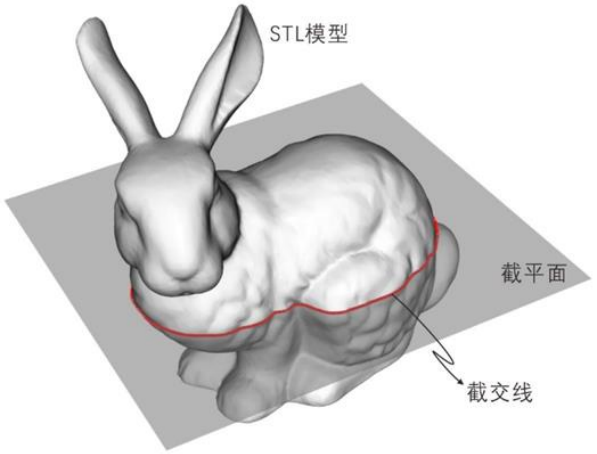
```
1   class StlModel:
2       def __init__(self):
3           self.triangles = []                                  # 三角面片列表
4           self.xMin = self.xMax = self.yMin = self.yMax = self.zMin = self.zMax = 0  # 模型边界
5       def getFacetNumber(self):                                # 获取STL模型中的面片数
6           pass
7       def getCoords(self, line):                               # 从文本中提取坐标，被readStlFile调用
8           pass
9       def readStlFile(self, filepath):                         # 读取STL文件，输入文件路径
10          pass
11      def extractFromVtkStlReader(self, vtkStlReader):         # 从vtkSTLReader提取面片信息
12          pass
13      def getBounds(self):                                     # 获取模型6个方向边界极值
14          pass
```

- StlModel类中，封装了两个STL文件读取函数，即**readStlFile**和**extractFromVtkStlReader**。函数readStlFile可用来读取文本形式的STL文件，只需输入STL文件路径即可；函数extractFromVtkStlReader可从一个vtkSTLReader类型的对象提取面片数据，因为vtkSTLReader本身可以处理文本或二进制形式的STL文件，因此如需读取二进制STL文件，也可使用这个函数。
- **getBounds**函数用于获取模型在X、Y、Z正负6个方向上的极值。

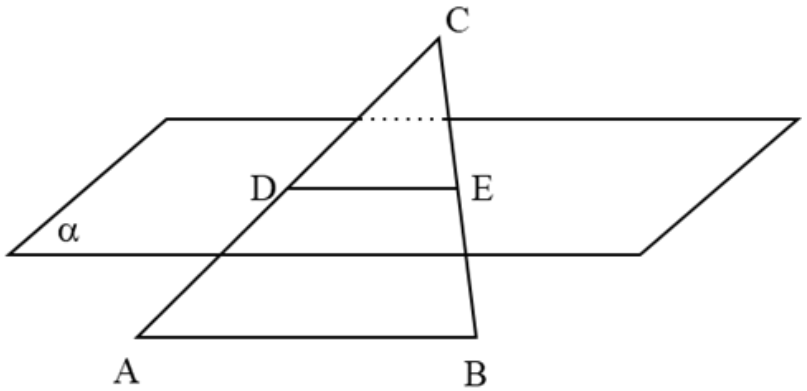
2. 空间三角形和平面求交算法

- **三维打印路径规划基本流程**

- （1）**STL模型切片**，就是用一系列平行平面（一般垂直于Z轴）去截模型，求取封闭交线。（2）**打印区域识别**，在同一平面上，若干交线之间可能存在包含嵌套关系，组成当前平面上的打印区域。（3）**填充路径生成**，在打印区域内规划填充路径，这些填充路径就是最后打印机喷头的运动轨迹。（4）**NC代码生成**，将不同区域、不同层之间的填充路径连接统一路径，并将这些路径坐标转化为打印机主控板可以识别的G代码。



平面截STL模型获取封闭交线



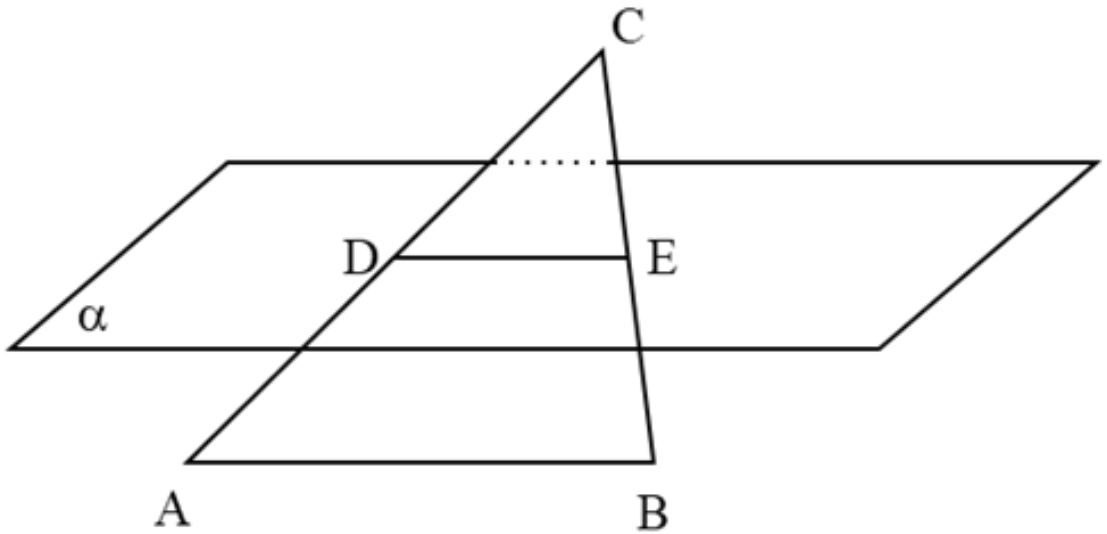
空间三角形和平面相交模型

- **STL模型切片是至关重要的一步。**这个问题可以描述为：用一张平面去截一个STL模型，计算该平面和模型之间的封闭交线。考虑到STL模型是由一系列离散的三角面片组成，因此计算平面和模型的交线首先需要**计算截平面和三角面片的交线（线段），**然后将这些交线段**依次首尾相连成封闭交线**即可。

2. 空间三角形和平面求交算法

• 问题和思路

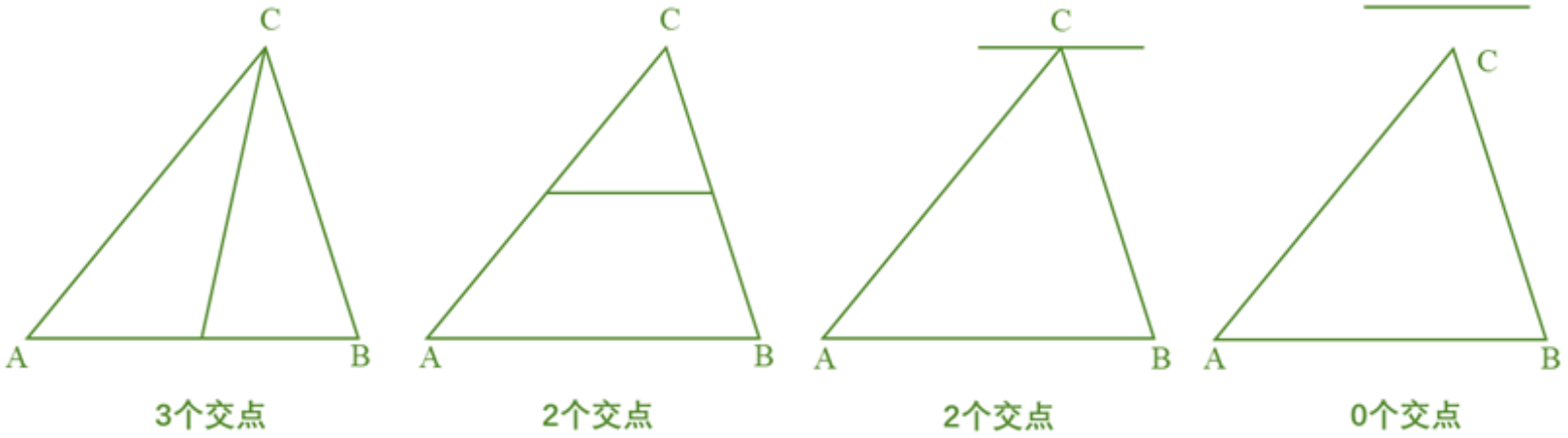
- 记空间中任意一三角形为ABC，平面为α(P，**N**)，其中P为平面经过的一点，**N**为平面法向量。ΔABC由3条边包围而成，分别为线段AB、AC和BC。
- 考虑到第2章已经推导了空间线段和空间平面的求交计算方法，并在GeomAlgo模块中给出了**intersectSegmentPlane**函数，结合这个函数，可以首先**分别计算线段AB、BC、AC和平面α的交点**，然后从中选择合适的**两点**作为ΔABC和平面α的交线。



空间三角形和平面相交模型

2. 空间三角形和平面求交算法

- 遵循以上思路，线段AB、BC、AC和平面α之间可能存在**3**个、**2**个或**0**个交点：
  - 如果交点个数为**3个**，那么其中必然有2个交点是三角形的同一顶点，这种情况下取3点中不重合的2个点作为交线两端点即可。
  - 如果交点个数为**2个**，则进一步判断这2个交点是否重合，如果不重合则取这2个点作为交线两端点，否则ΔABC和平面α无交线。
  - 如果交点个数为**0个**，则Δ BC和平面α无交线。



- **说明：**
  - 在intersectSegmentPlane函数中，如果线段和平面平行（或重合），认为线段和平面不存在交点；
  - 类似地，对三角形和平面，如果它们平行（或重合），则认为它们不相交；
  - 此外，如果三角形和平面交于三角形一个顶点，也认为它们不相交。

2. 空间三角形和平面求交算法

- **空间三角形和平面求交算法实现**
- 实现空间三角形和平面求交函数，由于这个函数依然涉及几何求交算法，因此将其写在GeomAlgo模块中，函数名为**intersectTrianglePlane**。

GeomAlgo.py	intersectTrianglePlane：空间三角形和平面求交函数	4-3

```
1   ……                                                  # 略过GeomAlgo.py前面的代码
2   def intersectTrianglePlane(triangle, plane):        # 输入为Triangle和Plane对象
3       AB = Segment(triangle.A, triangle.B)            # 三角形3条边
4       AC = Segment(triangle.A, triangle.C)
5       BC = Segment(triangle.B, triangle.C)
6       c1 = intersectSegmentPlane(AB, plane)           # 3条边和平面交点
7       c2 = intersectSegmentPlane(AC, plane)
8       c3 = intersectSegmentPlane(BC, plane)
9       if c1 is None:                                  # 分类讨论，枚举各种情况
10          if c2 is not None and c3 is not None:       # 存在2个交点的情况
11              if c2.distance(c3) != 0.0:
12                  return Segment(c2, c3)
13      elif c2 is None:
14          if c1 is not None and c3 is not None:       # 存在2个交点的情况
15              if c1.distance(c3) != 0.0:
16                  return Segment(c1, c3)
17      elif c3 is None:
18          if c1 is not None and c2 is not None:       # 存在2个交点的情况
19              if c1.distance(c2) != 0.0:
20                  return Segment(c1, c2)
21      elif c1 is not None and c2 is not None and c3 is not None:          # 存在3个交点的情况
22          return Segment(c1, c3) if c1.isIdentical(c2) else Segment(c1, c2)
23      return None                                     # 如果都不满足，返回None
```

注意

在设计算法时**切勿抱任何侥幸心理**，特别是涉及原理性的东西，以为某些情况可能不会发生而不去枚举相应应对措施，否则算法在后期测试或今后实际运行中总会碰到各种各样看来不可能发生的情况而奔溃。

2. 空间三角形和平面求交算法

- **空间三角形和Z平面求交**

- 考虑到在三维打印中，一般以Z轴作为模型生长方向，及截平面总是垂直于Z轴，可以再构建一个特化的函数，专门求取空间三角形和**Z平面**的交线。

GeomAlgo.py	intersectTriangleZPlane：空间三角形和z平面求交函数	4-4
1	……	# 略过GeomAlgo.py前面的代码
2	**def intersectTriangleZPlane(triangle, z):**	# 输入为Triangle对象和z高度
3	if triangle.zMinPnt().z > z:	# 预处理，如果三角形最低点高于
4	return None	# 平面或最高点低于平面，返回
5	if triangle.zMaxPnt().z < z:	# None，即无交线
6	return None	
7	return intersectTrianglePlane(triangle, Plane.zPlane(z))# 否则返回求交结果	

- **输入**：**Triangle对象**和**z平面高度（实数）**。

- 为了尽量减少计算量，在真正求交之前，先对三角形和平面的相对位置进行判断，如果三角形最低点仍然高于Z平面，那么返回None，同理如果三角形最高点仍然低于Z平面，也返回None。在排除这两种情况后，再调用空间三角形和平面求交的一般函数。

3. STL模型截交计算

- 在用一系列平行平面去截STL模型时，在每张平面上，都将得到一系列截交线段。为保存这些线段，需要建立一种数据结构把每个平面上的线段区分开，称这种数据结构为**Layer**（层），其属性如下：

Layer.py	切片层，保存层高、截交线段、截交线等信息	4-5

```
1  class Layer:
2      def __init__(self, z):
3          self.z = z                          # 当前层高度值
4          self.segments = []                  # 截交线段列表
5          self.contours = []                  # 多边形轮廓列表（封闭截交线）
```

- 基于Layer类和intersectTriangleZPlane函数，下面可以采用**暴力**的方法（brutal force）开始对STL模型进行截交计算：
  - 首先构造一系列平行平面，这个问题可以转化为**构造一系列层高值**。
  - 层高的最小值应在STL模型最低点上方，最大值应在模型最高点下方，而模型极值点可以通过StlModel类的getBounds函数获取。
  - 在每一层高度上，对STL模型中的所有面片进行遍历求交，并将交线保存到Layer对象中。

3. STL模型截交计算

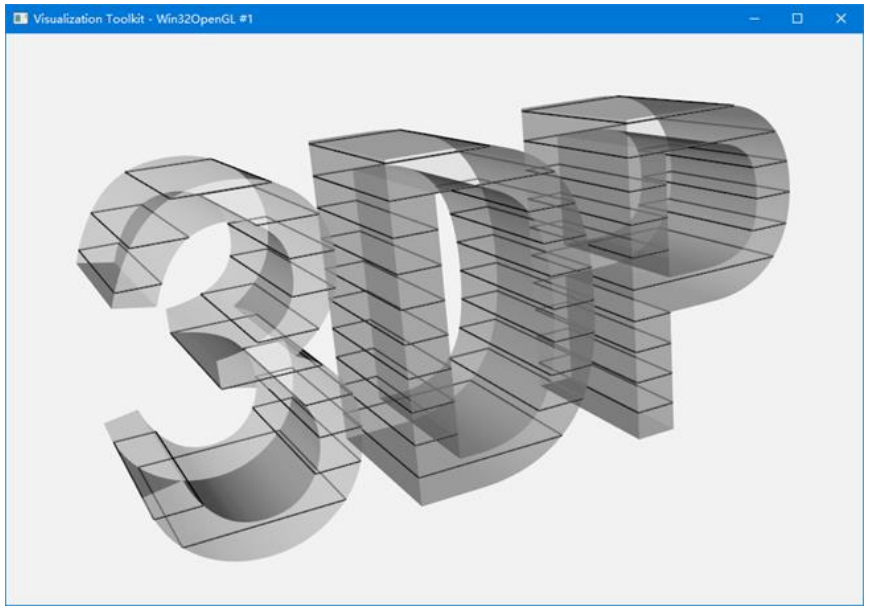
- 把暴力截交计算过程写在SliceAlgo.py文件中，建立**intersectStl_brutal**函数，具体代码如下：

SliceAlgo.py	intersectStl_brutal：对STL模型切片，得到截交线段，暴力法	4-6

```
1   ……                                                          # 导入必要的模块
2   def intersectStl_brutal(stlModel, layerThk):                # STL模型截交函数
3       layers = []                                             # 保存层对象列表
4       xMin, xMax, yMin, yMax, zMin, zMax = stlModel.getBounds()  # 模型极值
5       z = zMin + layerThk
6       while z < zMax:                                         # 截交主循环
7           layer = Layer(z)
8           for tri in stlModel.triangles:                      # 对当前高度，遍历所有三角形
9               seg = intersectTriangleZPlane(tri, z)           # 计算三角形和平面交线
10              if seg is not None:
11                  layer.segments.append(seg)                  # 将交线保存在layer中
12          layers.append(layer)
13          z += layerThk
14      return layers                                           # 返回层列表
15  if __name__ == '__main__':                                  # 截交函数测试
16      vtkAdaptor = VtkAdaptor()
17      vtkStlReader = vtk.vtkSTLReader()                       # 使用VTK读取STL模型
18      vtkStlReader.SetFileName("E:\\3DP.STL")
19      vtkAdaptor.drawPdSrc(vtkStlReader).GetProperty().SetOpacity(0.5)      # 显示模型
20      stlModel = StlModel()                                           # 建立StlModel对象
21      stlModel.extractFromVtkStlReader(vtkStlReader)          # 从VTK加载STL模型
22      layers = intersectStl_brutal(stlModel, 10.0)            # 暴力切片，层高为10.0mm
23      for layer in layers:                                    # 在VTK中显示所有截交线段
24          for seg in layer.segments:
25              segActor = vtkAdaptor.drawSegment(seg)
26              segActor.GetProperty().SetLineWidth(2)
27      vtkAdaptor.display()
```

3. STL模型截交计算

- 以上代码使用了较为简单的三维模型，这个模型包含3928个三角面片，Z向高度约为100mm，在层高设置为10mm的情况下，切片层数约为10层。
- 在每一层上，截平面要和3928个三角形求交；若考虑所有层，那么平面和三角形求交总次数约为39280次，这仅仅是层高在10mm的情况下。
- 考虑实际三维打印过程，将层高设置为0.1mm，那么平面和三角形总**求交次数将是39280000次**，这将消耗大量的时间。
- 对那些包含数十万、上百万三角面片的STL模型（事实上这很常见），求交的计算量更是惊人的！

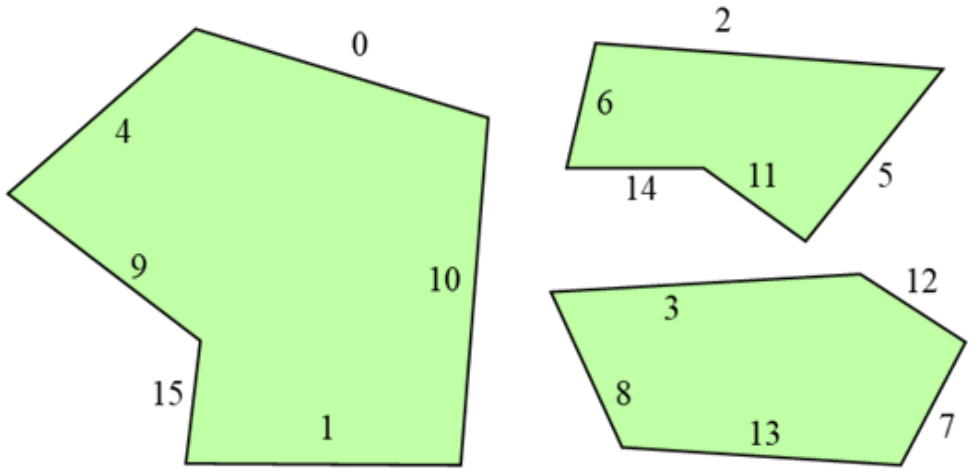


暴力截交结果

4. 截交线段拼接计算

• 问题和思路

- **问题描述**：对得到的某一层上的所有截交线段，如果把这些线段直接摆在平面上，它们将组成若干个封闭的多边形（轮廓线）。但事实上，由于STL模型中面片存储的无序性，这些截交线段在多边形中是毫无逻辑地**无序排放**的，比如线段列表中的1号线段可能落在这个多边形上，而2号线段可能并没有和1号线段首尾相连，而是落在了另一个多边形上。



截平面上无序排放的截交线段

- **暴力拼接基本思路**：首先从线段列表中拿出一条线段作为当前线段，然后**遍历**列表中剩余线段，找出一条和当前线段端点有重合线段，并将这条线段拼接到当前线段上。实现这个思路需要数据结构支撑——**Polyline**类可以直接使用。

4. 截交线段拼接计算

- **将平面上散乱线段集整理成多边形的步骤：**

1. 新建一条Polyline对象，记为contour。
2. 从线段集中取出一条线段，并将该线段从线段集移除，然后将该线段的两端点分别添加到contour。
3. 遍历线段集，将每条线段的两端点分别和contour的两端点比较：如果和contour终点重合，则将另一点添加到contour终点后；如果在contour起点重合，则将另一点添加到contour起点前。
4. 检查contour是否封闭：如果不封闭，则跳转至步骤3；否则保存contour，并检查线段集中是否还有线段剩余，如果有剩余则跳转至步骤1，否则结束流程。

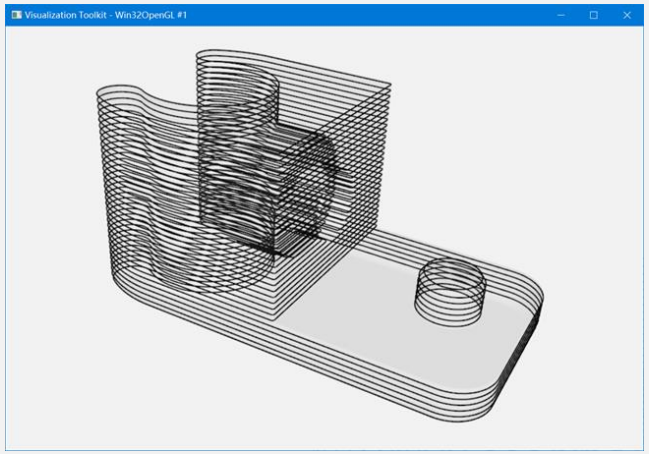
- 以上步骤将输出一系列封闭多边形，只要不停收集contour，并将contour添加到当前层（参见Layer类）的多段线列表（contours）中即可。

4. 截交线段拼接计算

- 封装将散乱线段集整理成多段线的函数，依然将这个函数写在SliceAlgo.py文件中，函数名为**linkSegs_brutal。**

SliceAlgo.py	linkSegs_brutal：将截交线段整理成封闭轮廓，暴力法	4-7

```
1   ……                                          # 略过Slicer.py前面的代码
2   def linkSegs_brutal(segs):                  # 定义暴力拼接函数
3       segs = list(segs)                       # 根据原列表建立新的列表，必要
4       contours = []                           # 最终得到的拼接轮廓contours
5       while len(segs) > 0:                    # 如果segs中还有元素
6           contour = Polyline()                # 新建轮廓并添加至contours
7           contours.append(contour)
8           while len(segs) > 0:
9               for seg in segs:                # 遍历搜索
10                  if contour.appendSegment(seg):  # 如果将seg添加至轮廓成功
11                      segs.remove(seg)        # 则从segs中移除seg
12                      break
13              if contour.isClosed(): break    # 如果轮廓闭合，退出当前循环
14      return contours                         # 最终返回封闭轮廓
15  if __name__ == '__main__':                  # 测试main函数
16      ……                                      # 省略代码，参考代码4-6
17      layers = intersectStl_brutal(stlModel, 1.0)  # 暴力切片，层高为1.0mm
18      for layer in layers:
19          layer.contours = linkSegs_brutal(layer.segments)  # 线段转多边形，暴力法
20          for contour in layer.contours:      # 显示各个多边形
21              polyActor = vtkAdaptor.drawPolyline(contour)
22              polyActor.GetProperty().SetLineWidth(2)
23      ……                                      # 省略代码，参考代码4-6
```

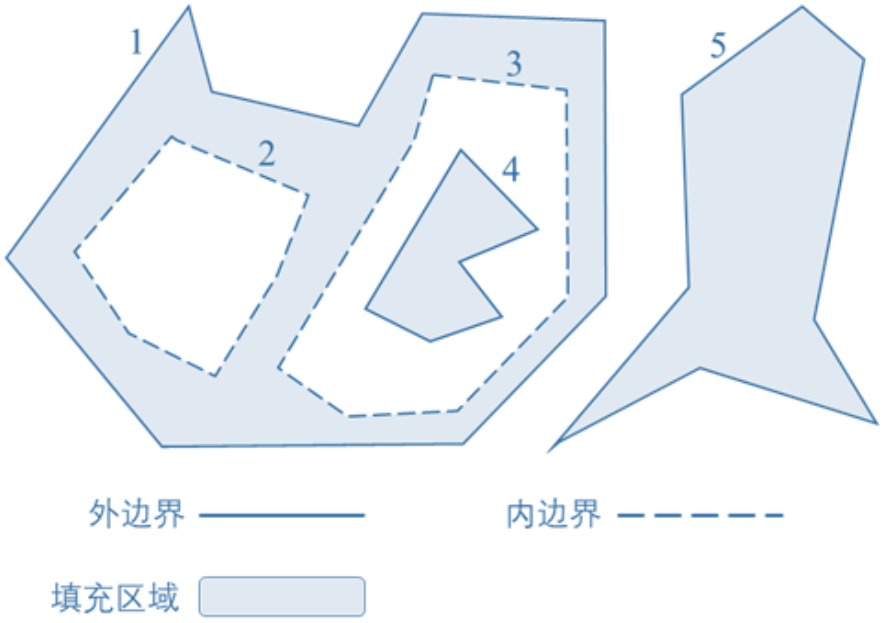


将截交线段拼接成封闭多边形（暴力法）

- **linkSegs_brutal**函数虽然总是能够返回正确的结果，但其时间复杂度为**O(n^2)**，呈n的平方量级，其中n为当前层上需要拼接的线段条数。

5. 轮廓方向识别与调整

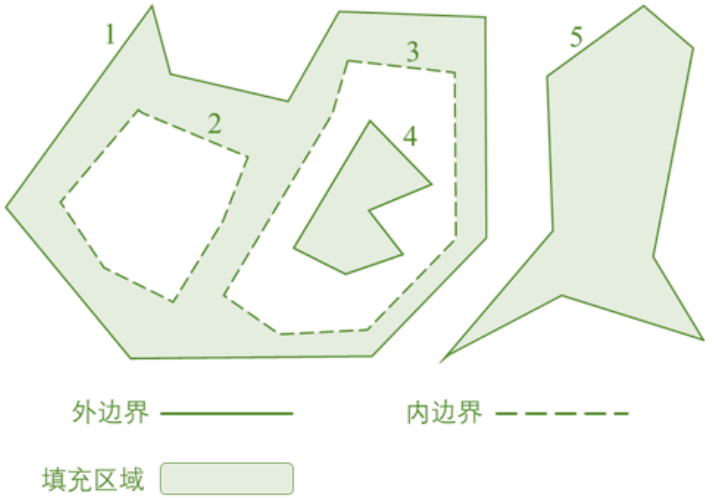
- 在执行以上拼接算法时并未考虑得到的封闭多边形的**方向**，因此它们的**方向完全是随机的**。
- 理论上，如果STL模型是合法的，这些多边形之间**不可能存在相交的情况**，它们在平面上的关系至多存在两种情形，**或者包含，或者不包含**，这两种情形相互组合，使得多边形在平面上共同形成打印填充区域的内、外边界。
- 为了区分内外边界，定义区域**外边界方向为逆时针，内边界方向为顺时针。**

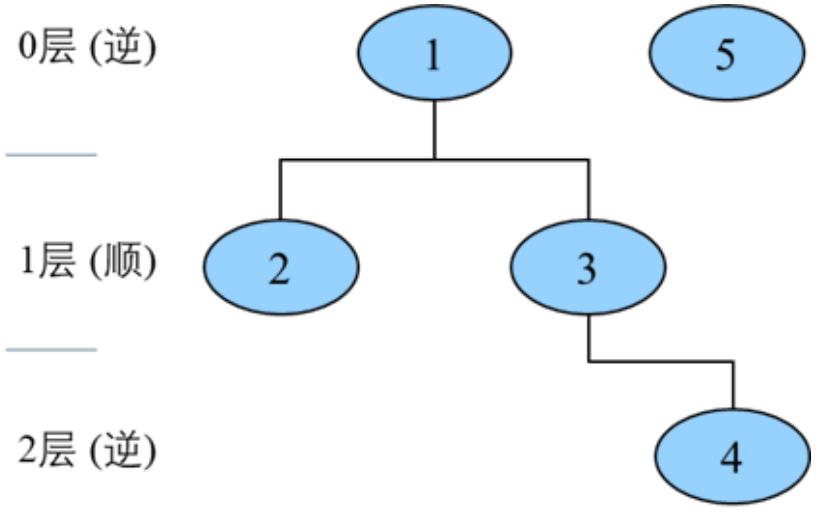


由多边形内外边界行程的填充区域

5. 轮廓方向识别与调整

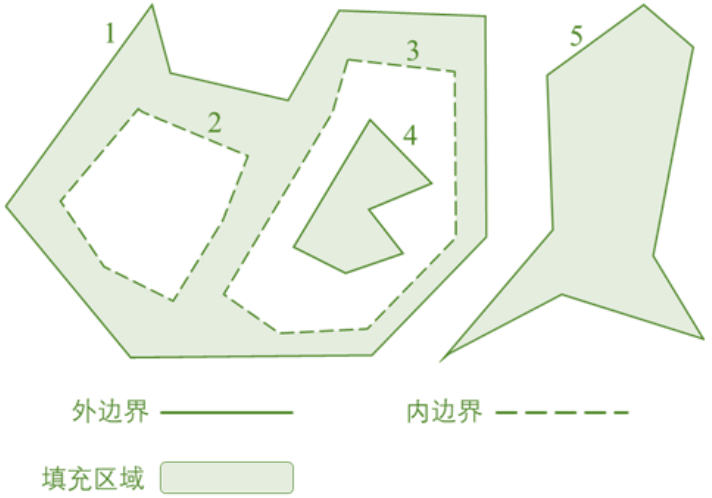
- **层次分析**
  - 如果一个多边形位于最外层（即第0层），那么这个多边形是填充区域的外边界，其方向应为逆时针，如图中1号和5号多边形；
  - 如果一个多边形处于第1层（即第0层内部），那么这个多边形是填充区域的内边界，其方向应为顺时针，如图中2号和3号多边形；
  - 如果一个多边形处于第2层，那么这个多边形又将是填充区域的外边界，其方向应为逆时针，如图中4号多边形。
  - ……
- **结论**：如果一个多边形处在的偶数层上，那么它为外边界，方向为逆时针；反之如果处在奇数层上，那么它为内边界，方向为顺时针。

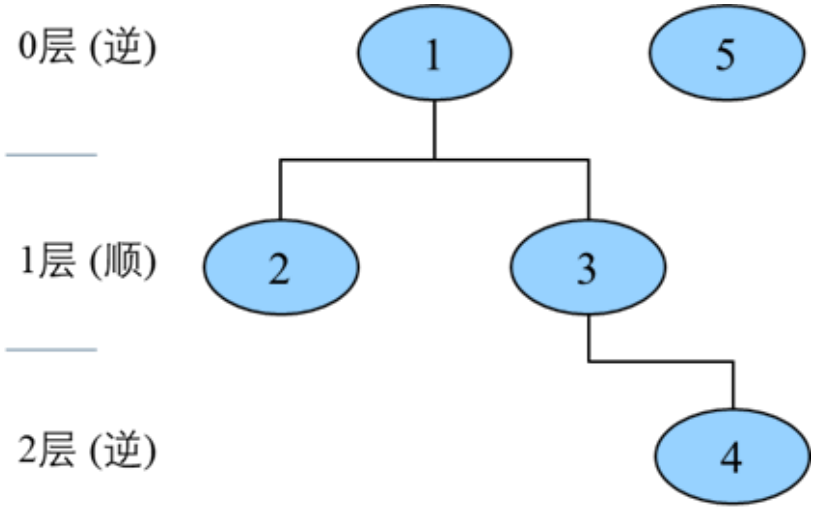




5. 轮廓方向识别与调整

- **父子关系定义**：对于两个多边形A和B，如果B位于A内部，那么称B为A的子多边形，A为B的父多边形。
  - 图中，1号和5号多边形均有0个父多边形，2号和3号多边形均有1个父多边形，4号多边形有2个父多边形。
- 由此不难归纳：如果一个多边形有**偶数个父多边形**，那么这个多边形为区域外边界，方向应为**逆时针**；反之如果一个多边形有**奇数个父多边形**，那么这个多边形为区域内边界，方向应为**顺时针**。
- **轮廓方向识别思路**：结合GeomAlgo模块的**pointInPolygon**函数，对于多边形集合中的任意多边形A，取该多边形上的任意一（顶）点P，判断P和剩余多边形的关系：如果P在偶数个多边形内部，那么多边形A应为外边界，方向应为逆时针；反之如果P在技术个多边形内部，那么多边形A应为内边界，方向应为顺时针。





5. 轮廓方向识别与调整

- 在GeomAlgo模块中添加调整多边形方向的函数**adjustPolygonDirs**，输入为Layer中的多边形轮廓列表。

GeomAlgo.py	实现adjustPolygonDirs函数：识别轮廓线顺、逆时针方向	4-8

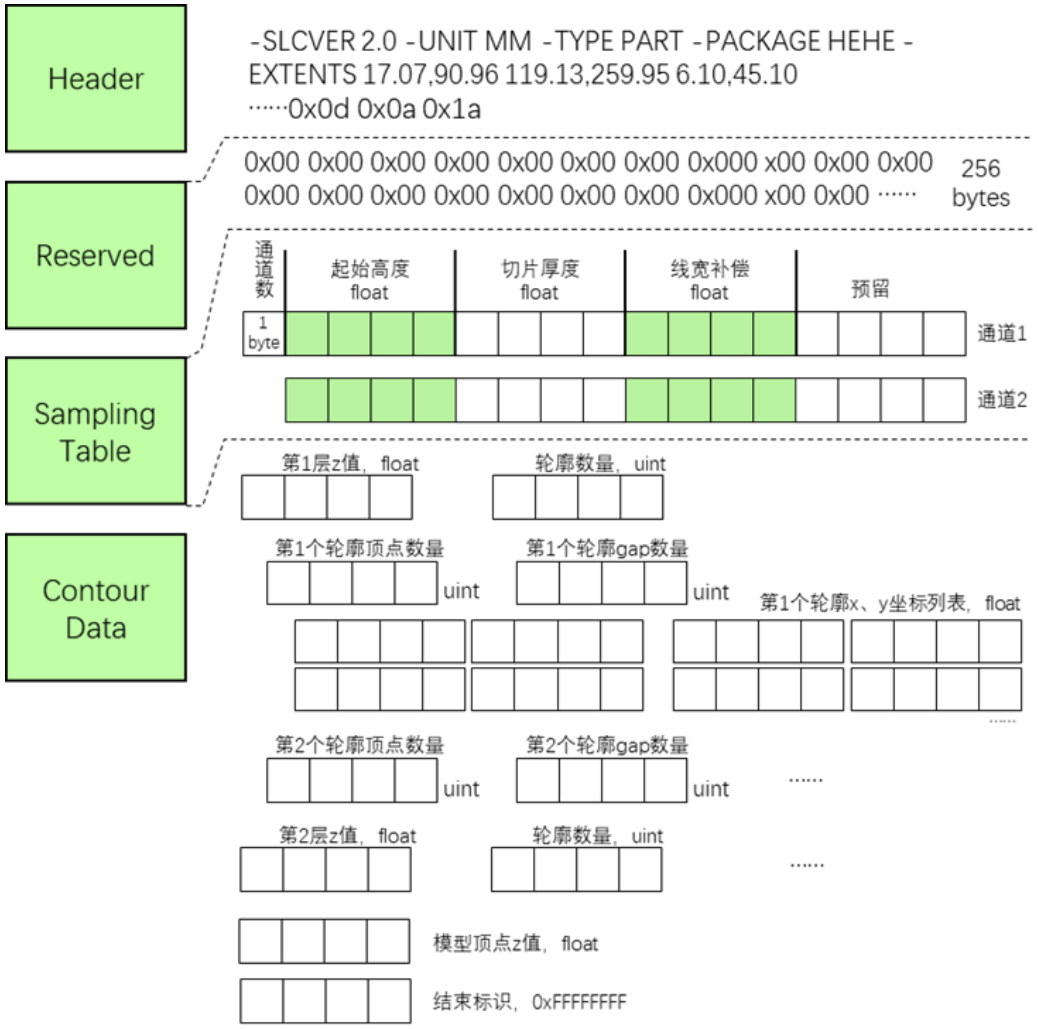
```
1   ……                                  # 略过Slicer.py前面的代码
2   def adjustPolygonDirs(polygons):    # 输入为轮廓列表
3       for i in range(len(polygons)):  # 第1个循环
4           pt = polygons[i].startPoint()   # 取出待检测轮廓上的起点
5           insideCount = 0             # 点在几个多边形内部计数
6           for j in range(len(polygons)):  # 第2个循环
7               if j == i: continue     # 如果两个多边形一样则跳过
8               restPoly = polygons[j]
9               if 1== pointInPolygon(pt, restPoly):    # 点在另一多边形内部，则加1
10                  insideCount += 1
11          if insideCount % 2 == 0:    # 判断点在内部次数是否为偶数
12              polygons[i].makeCCW()   # 调整多边形方向为逆时针
13          else:
14              polygons[i].makeCW()    # 调整多边形方向为顺时针
```

- **思考其他方法，用于识别多边形轮廓方向。**

6. 切片数据存取

- 通过暴力求交和暴力拼接，可实现对STL模型切片最终得到一系列切片数据layers，其中每个layer又包含一系列封闭的轮廓contours。为方便后续使用，需要对切片数据进行保存。
- **SLC文件**：和STL一样，SLC（SLiCe）文件格式同样由3D systems制定，如果说STL是三维打印模型文件格式的标准，那么SLC就是三维打印中切片数据文件格式的标准。SLC文件使用2.5维的轮廓线表示CAD模型，它有层的概念，层高随着Z方向增长，其中每层截面由内、外边界轮廓多边形组成。
- SLC文件由文本和二进制混合编码而成，包含4个部分：
  - 头文件（Header Section）
  - 预留区（Reserved Section）
  - 采样表（Sampling Table Section）
  - 轮廓数据区（Contour Data Section）

6. 切片数据存取



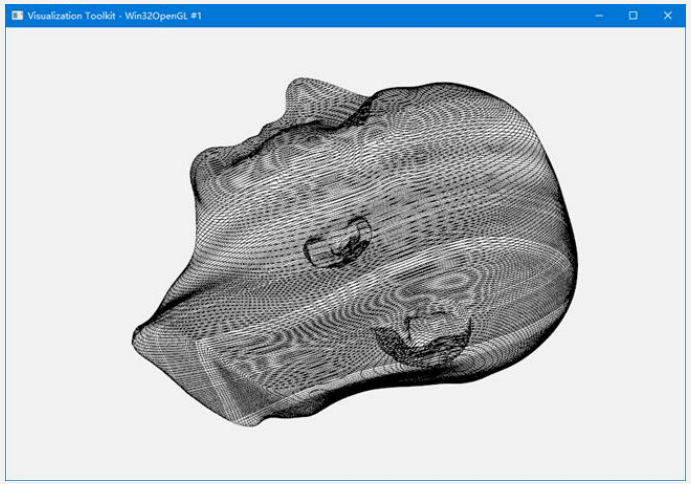
- **头文件**包含一系列ASCII文本键值对，指示了当前文件的全局信息，的SLC版本号（-SLCVER）、长度单位（-UNIT）、生成软件（-PACKAGE）、模型包络（-EXTENTS）等信息。头文件最大不能超过2048个字节，结束标识为0x0d（回车）、0x0a（换行）、0x1a（CTRL+Z）三个字符。
- **预留区**包含256个字节，是为将来扩展使用的预留空间，目前在这个区域内不定义任何东西，默认每个字节值为0。
- **采样表**由若干通道组成，开头第1个字节指示了通道数量，最大值为255。每个通道有4个字段，每个字段占4字节，共16个字节，分别为切片起始高度（float）、切片厚度（float）、线宽补偿（float）以及最后的保留字段。
- **轮廓数据区**用于保存多边形轮廓的坐标信息。该区域按层的形式组织轮廓。每层数据从当前层z值开始（4字节，float），随后是当前层轮廓数量（4字节，uint，整型）。紧接着是第1个轮廓顶点数（4字节，uint）和gap数（4字节，uint），然后是第1个轮廓x、y坐标列表，其中每个坐标（如x值）占4个字节，为float类型，x、y坐标总是成对出现。如果当前层还有第2个轮廓，那么后面接的是第2个轮廓的顶点数、gap数以及坐标列表。以此类推，直至罗列完当前层所有轮廓。在这之后，再接下一层轮廓数据。该区域结尾处有8个字节，前面4个字节用于标识当前模型的最高点z坐标，后面4个字节是固定为0xFFFFFFFF值，标识轮廓数据区到此结束，同时也标识整个文件到此结束。以上我们提到了gap，对某一轮廓，如果轮廓上某个位置存在两相邻点重合的情况，那么就说这个位置为gap，gap可以看作是轮廓线在此出现了断裂（沟）。

6. 切片数据存取

- 在SliceAlgo模块中实现两个函数，一个是写SLC文件(**writeSlcFile**)，一个是读SLC文件(**readSlcFile**)。

SliceAlgo.py	writeSlcFile：将切片数据保存为SLC文件	4-9

```
1   ……                                          # 略过Slicer.py前面的代码
2   def writeSlcFile(layers, path):             # 输入Layer列表以及文件名
3       f = None
4       try:
5           f = open(path, 'w+b')               # 打开（或创建）一个二进制文件
6           f.write(bytes("-SLCVER 2.0 -UNIT MM", encoding='utf-8'))  # 写入Header区
7           f.write(bytes([0x0d, 0x0a, 0x1a]))  # Header结尾3字节
8           f.write(bytes([0x00]*256))          # 写入Reserved区256个0x00
9           f.write(struct.pack('b', 1))        # 写入Sampling Table区
10          f.write(struct.pack('4f', 0, 0, 0, 0))
11          for layer in layers:                # 写入Contour Data区
12              f.write(struct.pack('fI', layer.z, len(layer.contours)))
13              for contour in layer.contours:
14                  f.write(struct.pack('2I', contour.count(), 0))
15                  for pt in contour.points:
16                      f.write(struct.pack('2f', pt.x, pt.y))
17          f.write(struct.pack('fI', layers[-1].z, 0xFFFFFFFF))        # 写入文件结尾
18      except Exception as ex:
19          print("writeSlcFile exception:", ex)        # 打印异常
20      finally:
21          if f: f.close()
```



7. SliceModel模块

- 使用**面向对象编程**的思路重新封装切片相关代码，建立表示切片模型的SliceModel类，实现STL模型加载、截交、拼接、方向调整，SLC文件存取等功能。

SliceModel.py	切片模型类，封装了对STL模型截交、拼接等算法	4-11

```
1   import SliceAlgo                                          # 导入SliceAlgo模块
2   class SliceModel:
3       def __init__(self, stlModel, layerThk, sliceAlgo = "brutal"):      # 初始化函数，输入模型、
4           self.stlModel = stlModel                                       # 层高以及切片方式
5           self.layerThk = layerThk
6           if sliceAlgo == 'brutal': self.slice_brutal()
7           elif sliceAlgo == 'optimal': self.slice_smart()
8       def slice_brutal(self):                               # 切片函数，brutal
9           self.layers = SliceAlgo.intersectStl_brutal(self.stlModel, self.layerThk)
10          for layer in self.layers:
11              layer.contours = SliceAlgo.linkSegs_brutal(layer.segments)
12              SliceAlgo.adjustPolygonDirs(layer.contours)
13      def slice_optimal(self): pass                         # 切片函数，优化，暂时未实现
14      def writeSlcFile(self, path):                         # 写SLC文件
15          SliceAlgo.writeSlcFile(self.layers, path)
16      def readSlcFile(self, path):                          # 读SLC文件
17          self.layers = SliceAlgo.readSlcFile(path)
18      def drawLayerContours(self, va, start=0, stop=0xFFFF, step=1, clr=(0,0,0), lineWidth=1):
19          pass
20  if __name__ == '__main__':                                        # SliceModel类测试main函数
21      vtkAdaptor = VtkAdaptor()
22      stlModel = StlModel()
23      stlModel.readStlFile("E:\\3DP.STL")                   # 读取STL文件
24      sliceModel = SliceModel(stlModel, 0.7)                # 建立切片模型，输入层高0.7mm
25      sliceModel.drawLayerContours(vtkAdaptor)              # 绘制切片轮廓
26      vtkAdaptor.display()
```

7. SliceModel模块

- SliceModel类的**__init__**函数输入参数分别为STL模型、切片厚度以及切片算法类型，其中切片算法类型是指brutal或optimal，即暴力法或优化法。在设计SliceModel时，力求简化切片流程，为此，直接将切片相关函数放在了其初始化函数中。
- 为了应对暴力切片和优化切片两种情况，进一步定义了**slice_brutal**和**slice_optimal**两个函数，其中slice_brutal函数主要调用了SliceAlgo模块中的带brutal后缀的几个函数，而slice_optimal函数只定义但尚未实现。
- **writeSlcFile**和**readSlcFile**函数，这两个函数分别直接调用SliceAlgo模块中的相应函数实现SLC文件的存取。
- **drawLayerContours**函数，用于绘制指定层上的切片轮廓。可以通过该函数相应参数指定需要绘制的层、轮廓颜色以及线条宽度等。这个函数需要从外部传入一个VtkAdaptor的对象va，事实上，SliceModel中所有轮廓数据都是通过va对象绘制的。这种在实体中封装绘制函数的方法大大简化外部绘图代码量。

本章小结

- **这一章尝试对一个STL模型进行切片，输出每层切片轮廓。**
  - 首先介绍了STL文件的数据存储格式，并封装了一个StlModel类从文件或VtkSTLReader加载三角面片数据。
  - 对STL模型的切片事实上可以分解为几个子问题：一是平面和三角形的求交问题，二是截交线段的拼接问题，三是轮廓线的方向识别和调整问题。针对这几个问题，分别介绍了相应算法，并在SliceAlgo模块中实现了相应的函数来解决这些问题。
  - 为了保存得到的切片轮廓数据，又介绍了SLC文件格式，并实现了SLC文件的存取。
  - 最后使用面向对象编程的逻辑，将上述切片功能统一封装在SliceModel类中。
- **在解决截交和拼接两个问题时，并未考虑算法执行效率，使用了暴力法，这是必要的过程，在想到更好的办法之前，暴力法是稳定有效的。**





思考题

1. 查阅资料，了解除STL外几种常用的三角面片模型存储格式，并比较各种格式优劣。
2. 实现一个函数，用于读取二进制形式的STL文件。
3. 尝试使用平面解析式计算截平面和三角面片的截交线段。
4. 代码4-7中，函数linkSegs_brutal有什么潜在问题？如何修改？
5. 尝试设计一种区别于代码4-8中adjustPolygonDirs的算法，用于识别平面上多个多边形见的包含关系及每个多边形的顺逆方向。
6. 什么是SLC文件？如何将STL文件转化为SLC文件？
7. 设计一种算法，尝试将SLC文件转化为STL文件。